Can we use AI with .NET?

A practical tour of SDKs, ergonomics, and the gotchas along the way.

AI with .NET

```
var process = Process.Start(new ProcessStartInfo
{
    FileName = "python",
    Arguments = "script.py",
    RedirectStandardOutput = true,
    UseShellExecute = false
})!;

Console.WriteLine(await process.StandardOutput.ReadToEndAsync());
await process.WaitForExitAsync();
```

Agenda

- Context & setup
- 7 demos: One task. Done 7 different ways.
- Gotchas & wrap-up

Disclaimer

~~"Is Copilot-generated C# actually good?"~~

~~"Should I let AI write all my .NET code?"~~

~~"Which model is the best?"~~

We're just here to make an API call. 7 times.

> **That said...** Check out 👉 bradygaster/squad

The prompt we'll use

Same question. Different tool each time.

> "Suggest 3 tech events with an AI focus that I can attend in Canada."

What we're comparing:

Setup complexity · Lines of code · Provider flexibility · Gotchas

GitHub: `link.reka.ai/dotnet`

Setup: .NET 10 file-based apps

No `.csproj`. No `.sln`. No `bin/` or `obj/`.

Each demo is a single `.cs` file — run it directly:

```
dotnet run 1-try-openai.cs
```

> Zero project friction. Clone → add API key → run.

> Pro tip: devcontainer included — works in GitHub Codespaces with zero local setup.

The tools — quick definitions

OpenAI SDK for .NET

Official SDK from OpenAI. Mirrors the API shape directly. Works with any OpenAI-compatible endpoint.

Microsoft.Extensions.AI

Microsoft's vendor-neutral abstraction layer. Defines `IChatClient` swap providers without changing your call logic.

Microsoft AI Agent SDK

Multi-language framework built with the best of `Semantic Kernal` and `AutoGen`. Very new - still in preview.

Raw HttpClient

No AI SDK. You speak directly to the REST API using .NET's built-in HTTP client. Maximum portability.

Demo 1 — OpenAI SDK

```
#:package OpenAI@2.8.0

var client = new OpenAIClient(
    new ApiKeyCredential(API_KEY),
    new OpenAIClientOptions { Endpoint = new Uri(BASE_URL) });

var chat = client.GetChatClient(MODEL);

var completion = await chat.CompleteChatAsync(
    new List<ChatMessage> { ChatMessage.CreateUserMessage(prompt) });

Console.WriteLine(completion.Value.Content[0].Text);
```

Demo 1 — OpenAI SDK · Key points

Familiar shape
If you've used the OpenAI SDK in any language, this feels the same.

OpenAI-compatible
Change `BASE_URL` → works with Reka, Ollama, Azure OpenAI, and more.

Vendor-shaped API
Your code is coupled to OpenAI's types: `ChatMessage`, `Content[0].Text`...

Good starting point
Best choice if you're just getting started and don't need flexibility yet.

Demo 2 — Microsoft.Extensions.AI

```
#:package Microsoft.Extensions.AI.OpenAI@10.1.1-preview.1.25612.2

IChatClient client = new OpenAIClient(
    new ApiKeyCredential(API_KEY),
    new OpenAIClientOptions { Endpoint = new Uri(BASE_URL) })
    .AsChatClient(MODEL);   // ← the one magic method

Console.WriteLine(await client.GetResponseAsync(prompt));
```

The entire call is one line. 23 lines total — the shortest file in the repo.

Demo 2 — Microsoft.Extensions.AI · Key points

`IChatClient` — the key abstraction

Typed as `IChatClient`, not `OpenAIClient`. Swap the provider without touching your call logic.

```
// Today: Reka
IChatClient client = new OpenAIClient(...).AsChatClient(MODEL);

// Tomorrow: Azure OpenAI — only this line changes:
IChatClient client = new AzureOpenAIClient(...).AsChatClient(MODEL);

// Your call code is identical either way:
Console.WriteLine(await client.GetResponseAsync(prompt));
```

> Great default for most apps. Low ceremony, high flexibility.

Demo 3 — Microsoft AI Agent SDK

```
#:package Microsoft.Agents.AI@1.0.0-preview.260209.1
#:package Microsoft.Extensions.AI.OpenAI@10.1.1-preview.1.25612.2

IChatClient chatClient = new OpenAIClient(...)
    .AsChatClient(MODEL);

AIAgent agent = new ChatClientAgent(
    chatClient,
    instructions: "You are a helpful assistant that recommends tech events.",
    name: "RekaAgent");

Console.WriteLine(await agent.RunAsync(prompt));
```

Demo 3 — Microsoft AI Agent SDK · Key points

Built on top of MEA — not a replacement.

What the Agent SDK adds:

- System instructions baked into the agent — not per-call
- `RunAsync` — a higher-level concept than "send a message"

> **Preview:** `1.0.0-preview.260209.1` — date-stamped versioning (Feb 9, 2026).

Demo 4 — Agent SDK + Structured Output

```
#pragma warning disable IL2026, IL3050  // ← we'll come back to this

var schema  = AIJsonUtilities.CreateJsonSchema(typeof(TechEventsResponse));
var options = new ChatOptions {
    ResponseFormat = ChatResponseFormat.ForJsonSchema(schema, strictEnabled: true)
};

var response = await agent.RunAsync(prompt, options);
var result   = response.Deserialize<TechEventsResponse>(jsonOptions);
foreach (var evt in result.Events)
    Console.WriteLine($"  - {evt.Name} | {evt.Location} | {evt.Date}");
```

```
public class TechEventsResponse { public List<TechEvent> Events { get; set; } }
public class TechEvent { public string Name, Location, Date { get; set; } }
```

Demo 4 — Structured Output · Key points

- **Define a C# type → get typed data back. No string parsing.**

`#pragma warning disable IL2026, IL3050`

`CreateJsonSchema()` uses reflection. In a trimmed/NativeAOT build this can break.

- **Provider portability**

```
var baseUrl = "https://api.reka.ai/v1";
// var baseUrl = "https://api.openai.com/v1";
// var baseUrl = "...anthropic.com/v1";
// var baseUrl = "http://localhost:11434/v1";
```

```
var model = "reka-flash-research";
var model = "gpt-5";
var model = "claude-opus-4-6";
var model = "llama3.1:8b";
```

Same code. Any OpenAI-compatible provider.

Demo 5 — OpenAI + Structured Output

```
#pragma warning disable OPENAI001  // experimental API

var client    = new ResponsesClient(MODEL, new ApiKeyCredential(API_KEY));
var webSearch = ResponseTool.CreateWebSearchTool(
    searchContextSize: ResponseWebSearchContextSize.High,
    allowedDomains: ["sessionize.com", "microsoft.com", "github.com"]);

var options = new ResponseCreationOptions();
options.Tools.Add(webSearch);

var response = await client.CreateResponseAsync(prompt, options);
Console.WriteLine(response.Value.GetOutputText());
```

> Only works with OpenAI.

Demo 5 — OpenAI Responses API · Key points

Power

- Real-time web search built in
- Domain allow-list for grounding
- `ResponsesClient` = newest OpenAI API
- Results can cite sources

Lock-in

- Proprietary — OpenAI only
- `OPENAI001` = experimental, can change
- No Reka, Ollama, or Azure support
- Trade: power for portability

Demo 6 — Raw HttpClient (no AI SDK)

```
#:package DotNetEnv@3.1.1   // only dependency

var httpClient = new HttpClient();
httpClient.Timeout = Timeout.InfiniteTimeSpan;         // ← don't skip this!
httpClient.DefaultRequestHeaders.Authorization =
    new AuthenticationHeaderValue("Bearer", API_KEY);

var request = new ChatRequest(MODEL, [new ChatMessage("user", prompt)]);
var json = JsonSerializer.Serialize(request, ChatRequestContext.Default.ChatRequest);
var resp = await httpClient.PostAsync(BASE_URL + "/chat/completions",
    new StringContent(json, Encoding.UTF8, "application/json"));

var doc = JsonDocument.Parse(await resp.Content.ReadAsStringAsync());
Console.WriteLine(doc.RootElement
    .GetProperty("choices")[0].GetProperty("message")
    .GetProperty("content").GetString());
```

Demo 6 — Raw HttpClient · Key points

important gotcha:

`httpClient.Timeout = Timeout.InfiniteTimeSpan;`

Default timeout is 100s. AI inference can take longer. It **silently** kills your request.

Source-generated JSON serialization (AOT-friendly):

```
[JsonSerializable(typeof(ChatRequest))]
[JsonSourceGenerationOptions(PropertyNamingPolicy = JsonKnownNamingPolicy.CamelCase)]
internal partial class ChatRequestContext : JsonSerializerContext { }
```

The punchline: Works with any OpenAI-compatible API. Forever. No SDK version to update.

Trade-off: you own the JSON, the headers, and the error handling. SDKs handle all of that for you.

Demo 7 — Raw HttpClient + Web Search + Structured Output

```
var requestJson = new JsonObject
{
    ["model"] = modelName,
    ["messages"] = new JsonArray(
        new JsonObject { ["role"] = "user", ["content"] = prompt }
    ),
    ["response_format"] = GetResponseFormat(),   // JSON schema
    ["research"] = new JsonObject
    {
        ["web_search"] = new JsonObject
        {
            ["allowed_domains"] = new JsonArray(
                "sessionize.com", "microsoft.com", "github.com")
        }
    }
};
var jsonPayload = requestJson.ToJsonString();
```

Demo 7 — Raw HttpClient (advanced) · Key points

Same zero-dependency approach as Demo 6 — but with Reka's advanced features.

Web search + domain filtering
Reka's `research.web_search` field grounds answers in live results from approved sites.

Structured output via JSON schema
No SDK support needed — just send the schema in `response_format` directly.

Gotcha: `JsonSerializerContext` disables reflection globally
Defining a `JsonSerializerContext` in the same assembly prevents `JsonSerializer.Serialize` from working on anonymous types. Fix: use `JsonObject` / `JsonNode` instead — no reflection needed.

Lessons & Gotchas

HttpClient.Timeout
The default 100s WILL silently kill slow AI responses. Set `InfiniteTimeSpan` or something reasonable.

Preview packages
MS Agent SDK and MEA.OpenAI are cutting-edge. Date-stamped versions. Expect breaking changes.

Pragma warnings are informative
`OPENAI001`, `IL2026`, `IL3050` — they're telling you something real. Don't blindly suppress them.

Lock-in is a spectrum
Raw HTTP = zero lock-in. Responses API = maximum. Know where you land before you build.

Compatible ≠ feature parity
Structured output, web search, tool calling — support varies by provider. Always test your target.

Which one should I use?

OpenAI SDK

- Just getting started
- OpenAI

MS AI Agent SDK

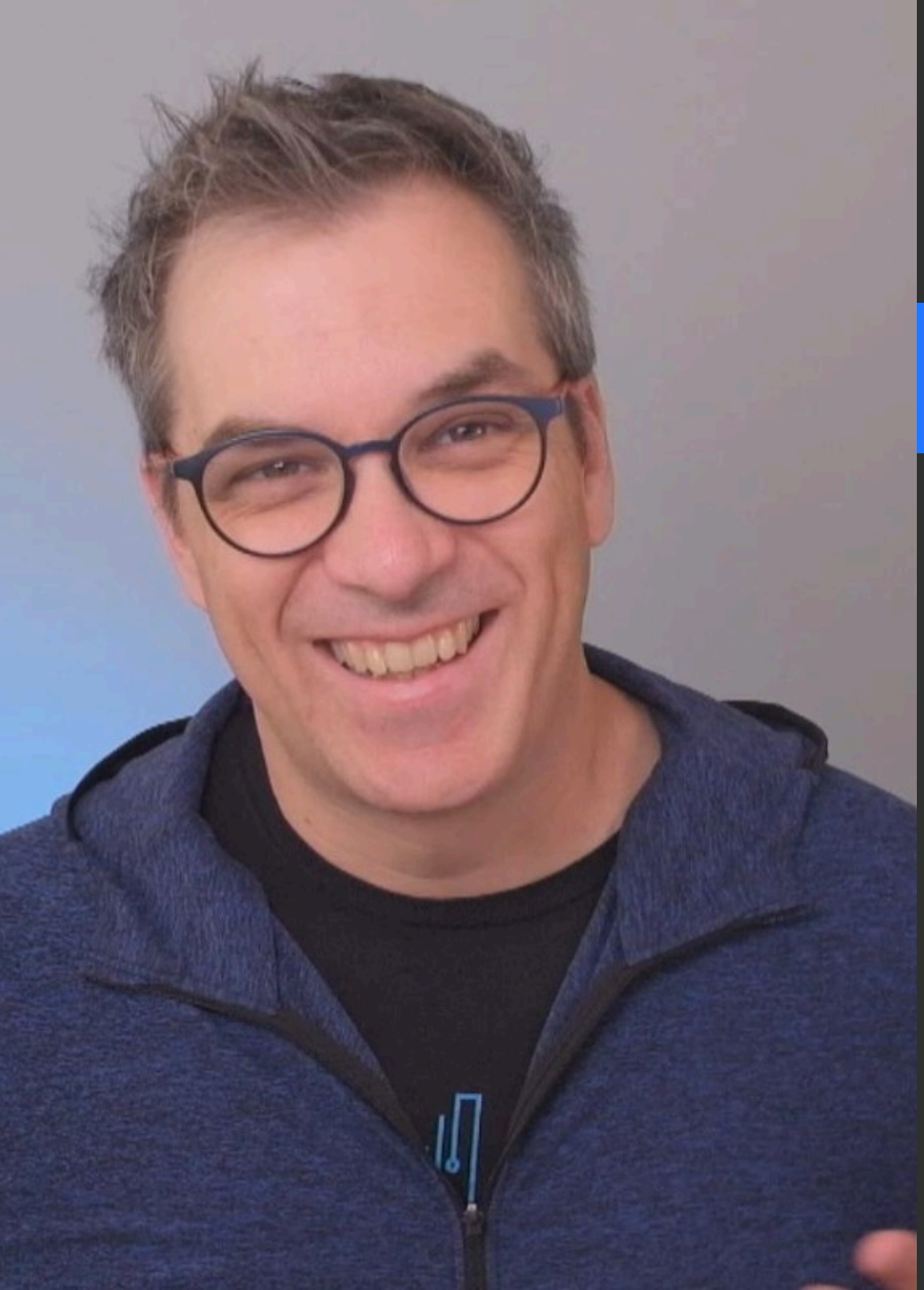
- Portability
- Documention
- Features

Raw HttpClient

- Everything is possible, today

link.reka.ai/dotnet

Code available here



Thank you

Frank Boucher

@fboucheros

fboucheros.com

reka

reka.ai